

Topic: Game Playing (Artificial Intelligence)

◆ 1. Game Playing কী?

Game Playing হলো Artificial Intelligence (AI)-এর একটি field যেখানে computer বা machine কে এমনভাবে design করা হয় যেন সে games (যেমন chess, tic-tac-toe) খেলতে পারে এবং **best possible move** নিতে পারে।

👉 এখানে goal হলো:

- Opponent কে হারানো
 - Optimal decision নেওয়া
-

◆ 2. Case Study: Grandmaster Chess (Deep Blue)

AI-এর game playing-এর সবচেয়ে famous example হলো **IBM Deep Blue vs Garry Kasparov**।

Deep Blue কী?

Deep Blue ছিল IBM-এর একটি supercomputer, যা chess খেলতে পারত।

Historical Event:

- 1996: Deep Blue প্রথমবার Garry Kasparov কে হারায় (একটি game)
- 1997: Deep Blue full match জিতে নেয়

👉 Match result:

- Kasparov: 1 win
- Deep Blue: 2 wins
- Draw: 3

📌 এটা ছিল AI ইতিহাসে একটি milestone, কারণ প্রথমবার একটি computer বিশ্বের best chess player কে defeat করে।

⚙️ Deep Blue কীভাবে কাজ করত?

Deep Blue মানুষের মতো intuition ব্যবহার করেনি। এটি ব্যবহার করত:

👉 Brute Force Approach

- সব possible moves calculate করা
- অনেক গভীর পর্যন্ত game tree explore করা
- fast computation + large memory ব্যবহার

📌 অর্থাৎ, এটি “smart thinking” না, বরং “massive calculation power” ব্যবহার করত।

◆ 3. Game Playing as Search Problem

Game playing কে AI তে সাধারণত **search problem** হিসেবে দেখা হয়।

👉 এখানে:

- Initial state = game এর starting position
- Actions = possible moves (edge)
- State space = সব possible board positions
- Goal = winning state

📌 AI একটি **game tree** তৈরি করে যেখানে:

- Nodes = game states
 - Edges = moves
-

◆ 4. Optimal Decisions in Games

AI-এর goal হলো এমন move নেওয়া যা **optimal outcome** দেয়।

👉 Optimal decision মানে:

- নিজের জন্য maximum advantage
- opponent এর best response consider করা

📌 এজন্য AI assumes করে:

Opponent-ও best possible move নেবে।

◆ 5. Greedy Search Algorithm

Greedy search হলো এমন একটি method যেখানে:

👉 প্রতিটি step-এ locally best move নেওয়া হয়।

Idea:

- ভবিষ্যৎ না ভেবে শুধু immediate best move choose করা

📌 Example:

Chess-এ যদি তুমি শুধু opponent-এর queen capture করার move choose করো, কিন্তু পরে checkmate হয়ে যাও—তাহলে greedy approach ভুল হবে।

👉 Limitation:

- Future consequences consider করে না
 - Always optimal solution দেয় না
-

◆ 6. Minimax Algorithm

Minimax হলো game playing-এর core algorithm।

👉 এটি দুই player game-এর জন্য ব্যবহার হয়:

- Player MAX → নিজের score maximize করে
- Player MIN → opponent minimize করে

Concept:

- MAX player tries to maximize utility
- MIN player tries to minimize it

👉 AI assumes opponent also plays optimally।

🧠 Example:

ধরি একটি simple game tree:

- MAX chooses a move
- MIN responds
- Then MAX আবার move নেয়

👉 Minimax:

- Leaf nodes এ utility value calculate করে
- তারপর backtracking করে best move নির্বাচন করে

📌 Final goal:

MAX এমন move choose করবে যাতে worst-case scenario-তেও best outcome থাকে

◆ 7. Alpha-Beta Pruning

Minimax algorithm অনেক large tree explore করে, যা slow হতে পারে।

👉 তাই ব্যবহার করা হয় **Alpha-Beta Pruning**

Idea:

- Unnecessary branches cut করে দেওয়া হয়
- যেগুলো final decision affect করবে না

Benefits:

- Search time কমে যায়
 - Efficiency বাড়ে
-

🔑 **Terms:**

- **Alpha (α)** → MAX-এর best value so far
- **Beta (β)** → MIN-এর best value so far

👉 যদি কোনো branch এ:

- $\alpha \geq \beta$ হয় → সেই branch prune করা হয় (skip)

📌 কারণ ঐ branch আর optimal decision-এ impact করবে না।



Summary

- Game playing in AI is treated as a **search problem**
 - Goal: **optimal decision making**
 - Deep Blue is a famous example of brute force-based AI
 - Greedy search → immediate best move
 - Minimax → considers opponent's optimal moves
 - Alpha-Beta pruning → improves Minimax efficiency
-

Game Playing and AI

◆ কেন Game Playing AI Research-এর জন্য গুরুত্বপূর্ণ?

Game playing AI-এর জন্য একটি **ideal problem domain**, কারণ এতে অনেক গুরুত্বপূর্ণ concept একসাথে পাওয়া যায়।

✓ 1. Game playing is non-trivial (সহজ না)

👉 “Non-trivial” মানে হলো সহজ বা straightforward না।

📌 Example:

- Tic-Tac-Toe সহজ মনে হলেও সব possible move চিন্তা করলে complexity বাড়ে
- Chess বা Go → extremely difficult

👉 তাই AI কে এখানে smart decision নিতে হয়।

✓ 2. Players need “human-like intelligence”

Game খেলতে হলে দরকার:

- Planning
- Strategy
- Prediction
- Decision making

👉 এগুলো সবই মানুষের intelligence-এর অংশ।

📌 Example:

Chess-এ তুমি শুধু move দাও না—

- opponent কী করবে সেটা আগে ভাবো
- future planning করো

👉 AI-কে এই “human-like thinking” imitate করতে হয়।

✓ 3. Games can be very complex

📌 Example:

- Chess → অনেক rules + huge possibilities
- Go → even more complex

👉 এই complexity AI research-এর জন্য perfect test environment।

✓ 4. Requires decision making within limited time

👉 Game-এ time constraint থাকে।

📌 Example:

Chess match-এ:

- তোমার কাছে limited time থাকে move দেওয়ার জন্য

👉 AI-কে fast + accurate decision নিতে হয়।

✓ 5. Game playing = Search Problem

👉 Game-কে আমরা search problem হিসেবে represent করতে পারি।

এখানে:

- **States** (অবস্থা) → board configuration
 - **Edges/Actions** (পথ) → legal moves
 - **Initial State** (শুরু) → starting board
 - **Goal State** (লক্ষ্য) → win / draw / lose
-

✓ 6. Game Tree (গেম ট্রি)

👉 Game-এ সব possible move নিয়ে একটি tree বানানো হয়।

📌 এখানে:

- Nodes → board অবস্থাগুলো
- White nodes → এক player (ধরি MAX)
- Black nodes → অন্য player (MIN)

👉 দুইজন player alternate turn নেয়।



Example (Simple Idea)

ধরি Tic-Tac-Toe:

- Root → empty board
- First level → সব possible first moves
- Second level → opponent moves

👉 এভাবে একটি বড় **game tree** তৈরি হয়।



Game Playing as Search

👉 আমরা দুইজন player-এর game consider করি:

- Tic-Tac-Toe
 - Checkers
 - Chess
-



Representation

Concept	Meaning
States	Board configurations

Edges	Legal moves
Initial State	Starting board
Goal State	Winning/terminal state

! New Aspect (নতুন বিষয়)

👉 সাধারণ search problem থেকে এটি আলাদা কেন?

কারণ:

এখানে একটি opponent আছে, যাকে আমরা control করতে পারি না !

🤔 Problem:

👉 যদি opponent unpredictable হয়?

👉 তখন AI কীভাবে handle করবে?

✓ Solution Idea:

👉 Assume করা হয়:

Opponent-ও **optimal move** নেবে

📌 এই assumption ব্যবহার করে Minimax algorithm কাজ করে।

◆ Game Playing as Search: Complexity

Complexity কী?

👉 Complexity মানে হলো:

- কতগুলো states explore করতে হবে
 - কত সময় লাগবে
-

Worst-case complexity:

👉 Formula:

$$O(b^d)$$

Where:

- **b = branching factor** (প্রতি node-এ কত move সম্ভব)
 - **d = depth** (game কত move পর্যন্ত যায়)
-

Example 1: Tic-Tac-Toe

- $b \approx 5$
- $d = 9$

👉 Total states $\approx$

$$5^9 = 1,953,125$$

 অর্থাৎ প্রায় 20 লাখ possible states!

Example 2: Chess

- $b \approx 35$
- $d \approx 100$

👉 Total states $\approx$

$$35^{100} \approx 10^{154}$$

😬 এটা বিশাল সংখ্যা!

👉 কিন্তু actual legal states $\approx 10^{40}$

⚠️ Key Insight

👉 Even then:

Search space is extremely huge !

📌 তাই:

- সব move brute force করা সম্ভব না
 - smart algorithms দরকার (Minimax, Alpha-Beta)
-



Summary (Exam-এর জন্য Important)

✓ Game playing AI research-এর জন্য গুরুত্বপূর্ণ কারণ:

- Non-trivial problem
- Human-like intelligence দরকার
- Highly complex
- Time-constrained decision

✓ Game = Search Problem:

- States = board
- Moves = edges
- Goal = win

✓ New challenge:

- Opponent exists (uncontrollable)

✓ Complexity:

- $O(b^d)$
- Chess $\rightarrow$ extremely large search space

Greedy Search in Game Playing

◆ 1. Utility Function কী?

👉 **Utility function** হলো এমন একটি function যা game-এর প্রতিটি **terminal state** (শেষ অবস্থা)-কে একটি numeric value (সংখ্যা) দেয়।

📌 এই value বোঝায়:

- সেই state টা computer-এর জন্য কতটা ভালো বা খারাপ

Utility Value Interpretation

Situation	Utility Value
-----------	---------------

Win (জয়)	Positive (+)
-----------	--------------

Loss (হার)	Negative (-)
------------	--------------

Draw (ড্র)	0
------------	---

📌 Typical range:

- $-\infty$ to $+\infty$
- বা -1.0 to +1.0

👉 বেশি value = better for computer

👉 কম value (negative) = better for opponent

◆ 2. Greedy Search কী করে?

👉 Greedy search decision নেয় এইভাবে:

1. প্রতিটি possible move consider করে
2. সেই move থেকে **terminal states** পর্যন্ত **expand** করে
3. প্রতিটি terminal state evaluate করে (utility function দিয়ে)
4. যে move-এর utility সবচেয়ে বেশি → সেটাই choose করে



- Game tree তৈরি করে
- সব branches expand করে terminal node পর্যন্ত
- Utility value calculate করে
- Maximum value যেটায় পাওয়া যায় $\rightarrow$ সেটি select করে

ধরি

- Move A $\rightarrow$ utility = +0.5
- Move B $\rightarrow$ utility = +0.8
- Move C $\rightarrow$ utility = -0.2



♦ 3. Greedy Search-এর Assumption

👉 এটি ধরে নেয়:

শুধু current best outcome দেখলেই যথেষ্ট

📌 অর্থাৎ:

- Future consequences ignore করে
 - Opponent কী move নেবে সেটা consider করে না
-

! 4. Greedy Search-এর Problem

👉 সবচেয়ে বড় limitation:

It ignores what the opponent might do!

! Problem Explanation:

Greedy search শুধু দেখে:

- আমার current move এর best outcome কী

কিন্তু ignore করে:

- opponent এর response
-

🎯 Example:

ধরি computer move নেয় C:

- Computer ভাবে → C best move (utility বেশি)

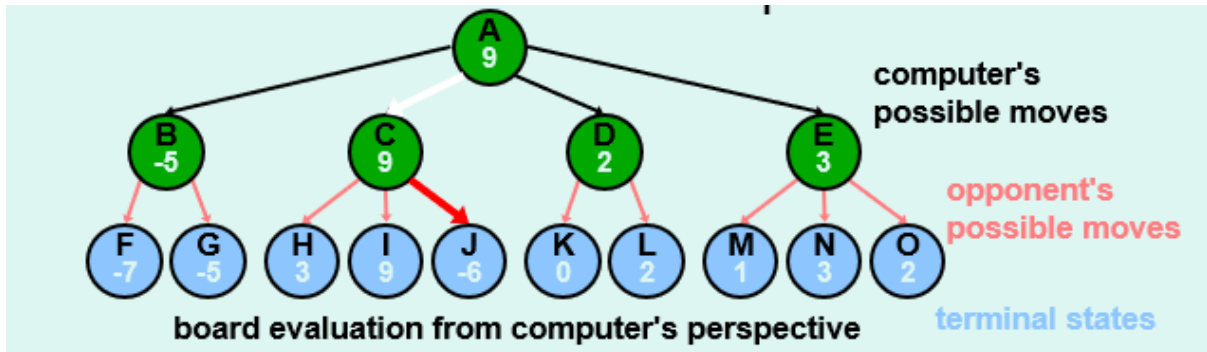
কিন্তু:

👉 opponent পরে move নিয়ে (J):

- Computer কে defeat করে দেয়

📌 তাই final outcome:

- Greedy অনুযায়ী ভালো মনে হলেও
- বাস্তবে game lose হয়ে যায়



🧠 Key Insight:

- 👉 Greedy search = short-sighted approach
- 👉 It does NOT plan ahead

▼ Summary (Exam Ready)

✓ Utility function:

- Terminal state কে numeric value দেয়
- Win → positive
- Loss → negative
- Draw → 0

✓ Greedy search:

- Each move evaluate করে
- Maximum utility move select করে

✓ Process:

- Expand → Evaluate → Select max value

! Limitation:

- Opponent-এর moves consider করে না
- Future consequences ignore করে

- তাই always optimal result দেয় না
-

- 👉 এজন্য Greedy search alone game playing-এর জন্য sufficient না
 - 👉 তাই Minimax algorithm ব্যবহার করা হয় (next topic)
-

Minimax: Idea

◆ Basic Concept

- 👉 Minimax algorithm game playing-এর একটি fundamental technique যেখানে:

AI ধরে নেয় opponent সবসময় **optimal (best) move** নেবে।

📌 অর্থাৎ:

- Computer নিজের জন্য best move খোঁজে
 - কিন্তু সাথে সাথে worst-case scenario-ও consider করে
-

◆ 1. “Assuming the worst” কী?

- 👉 Minimax-এর main assumption হলো:

Opponent সবসময় intelligent এবং optimal play করবে

📌 তাই AI এমন move choose করে যা:

- opponent-এর best response consider করার পরেও best outcome দেয়
-

◆ 2. Two Players Concept

- 👉 Game-এ দুই ধরনের player থাকে:

- **MAX (Computer)** → utility maximize করে AI (eikhane sobsomoy amra AI er perspective e chinta korbo)
 - **MIN (Opponent)** → utility minimize করে
-



Utility Interpretation

- High utility → computer-এর জন্য ভালো
 - Low utility → opponent-এর জন্য ভালো
-

◆ 3. Computer কী করবে?

👉 Computer (MAX player) সবসময়:

➡ **Maximizing moves choose** করবে

📌 কারণ:

- সে চায় utility value maximum হোক
-

◆ 4. Opponent কী করবে?

👉 Smart opponent (MIN player):

➡ **Minimizing moves choose** করবে

📌 কারণ:

- সে চায় computer-এর utility কমিয়ে দিতে
-

◆ 5. Minimax-এর Core Idea

👉 Computer move করার সময় শুধু নিজের move না দেখে:

নিজের move + opponent-এর best response → এই দুইটা consider করে decision নেয়



Step-by-step logic:

1. Computer একটি move করে
2. Assume করে opponent best possible counter-move করবে

3. সেই worst-case outcome evaluate করে
 4. তারপর সব moves compare করে best one select করে
-

◆ 6. Example (Conceptual)

ধরি 2-step game:

Level 1: Computer (MAX)

- Move A
- Move B

Level 2: Opponent (MIN)

- প্রতিটি move-এর পরে opponent minimizing move নেবে
-

Suppose utility values:

- Move A → opponent reduces to 3
- Move B → opponent reduces to 5

👉 Minimax decision:

- MAX wants maximum of these minimums

So:

- A → min = 3
- B → min = 5

👉 Computer chooses:

➡ Move B (because $5 > 3$)

◆ 7. “Values backed up in minimax way”

👉 Minimax-এ values leaf nodes থেকে root পর্যন্ত backtrack করা হয়।

📌 Process:

1. Terminal nodes এ utility calculate করা হয়
2. MIN level → minimum value নেয়
3. MAX level → maximum value নেয়
4. এভাবে values “back up” হয় tree-এর উপরে

🧠 Key Idea:

- MIN level → min value propagate
- MAX level → max value propagate

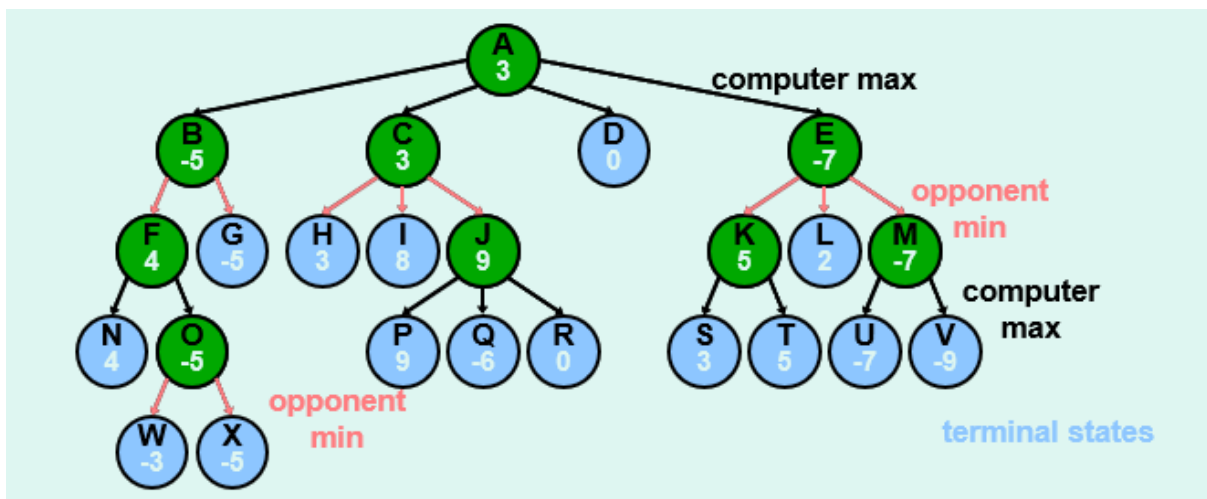
♦ 8. Generalization (>2 moves)

👉 Minimax শুধু 2 moves না, multiple levels পর্যন্ত apply করা যায়:

- Depth d পর্যন্ত game tree expand করা হয়
- Alternating levels:
 - MAX → MIN → MAX → MIN → ...

📌 Every level alternates between:

- Maximizing player
- Minimizing player





Summary (Exam-ready)

- ✓ Minimax assumes opponent plays optimally
- ✓ Two players:

- MAX (computer) → maximize utility
- MIN (opponent) → minimize utility

- ✓ Computer chooses move considering:

- Its own move
- Opponent's best response

- ✓ Process:

- Evaluate terminal nodes
- Back up values:
 - MIN → minimum
 - MAX → maximum

- ✓ Final decision:

Choose move with the best worst-case outcome



Minimax: Direct Algorithm

◆ 1. Basic Idea

👉 Minimax algorithm ব্যবহার করে computer তার **best move** খুঁজে বের করে, ধরে নিয়ে যে opponent সবসময় best (optimal) move নেবে।

◆ 2. Algorithm Steps (Step-by-Step)

👉 Computer যখন move নিতে যাবে, তখন এটি নিচের ধাপগুলো follow করে:

For each move by the computer:

1. Perform depth-first search to a terminal state
2. Evaluate each terminal state
3. Propagate upwards the minimax values
 - if opponent's move minimum value of children backed up
 - if computer's move maximum value of children backed up
4. choose move with the maximum of minimax values of children

Note:

- minimax values gradually propagate upwards as DFS proceeds: i.e., minimax values propagate up in "left-to-right" fashion
- minimax values for sub-tree backed up "as we go", so only $O(bd)$ nodes need to be kept in memory at any time

✓ Step 1: Depth-First Search (DFS)

👉 প্রতিটি possible move-এর জন্য:

➡ **Depth-First Search (DFS)** ব্যবহার করে tree-এর শেষ পর্যন্ত যায় (terminal state পর্যন্ত)

📌 DFS মানে:

- এক path ধরে যতদূর সম্ভব নিচে নামা
- তারপর backtrack করা

✓ Step 2: Evaluate Terminal States

👉 যখন terminal state (win/lose/draw) পাওয়া যায়:

➡ Utility function দিয়ে value assign করা হয়

📌 Example:

- Win $\rightarrow +1$
- Loss $\rightarrow -1$
- Draw $\rightarrow 0$

✓ Step 3: Propagate Values Upwards

👉 এখন leaf node থেকে value উপরে (root-এর দিকে) পাঠানো হয়

📌 Rules:

- **Opponent (MIN)** → children-এর minimum value নেয়
 - **Computer (MAX)** → children-এর maximum value নেয়
-

✓ Step 4: Choose Best Move

👉 Root node-এ এসে:

➡ যে move-এর minimax value সবচেয়ে বেশি → সেটাই choose করা হয়

◆ 3. “Values propagate upwards”

👉 Minimax-এ values নিচ থেকে উপরে আসে (backtracking)

📌 Important point:

- এটি **left-to-right DFS order**-এ calculate হয়
- subtree-এর value সাথে সাথে calculate করে up করে

👉 তাই পুরো tree memory-তে রাখতে হয় না

◆ 4. Space Complexity

👉 কারণ DFS ব্যবহার করা হয়:

Space Complexity = $O(bd)$

Where:

- b = branching factor
- d = depth

📌 কারণ:

- একসাথে পুরো tree না, শুধু current path memory-তে থাকে
-

◆ 5. Time Complexity

Time Complexity = $O(b^d)$

👉 কারণ:

- প্রতিটি node explore করতে হয়

📌 Example:

Chess-এ:

- $b \approx 35$
- $d \approx 100$

😱 → Extremely large!

! Key Problem

👉 Time complexity খুব বেশি!

Computer-এর কাছে limited time থাকে move নেওয়ার জন্য

👉 তাই full minimax practically possible না !

◆ 6. Practical Solution: Depth-Limited Search

👉 তাই আমরা করি:

➡ পুরো tree না গিয়ে, একটা fixed depth (m) পর্যন্ত search করি

📌 এটাকে বলে:

Depth-limited Minimax

Example:

- Full depth = 100 (chess) ✗
- Instead → $m = 3$ বা 4 ✓

👉 Faster decision নেওয়া যায়

◆ 7. New Problem (Very Important !)

👉 যদি আমরা depth m -এ stop করি:

তখন আমরা terminal state-এ নেই !

! Problem:

👉 Utility function তো শুধু terminal state-এর জন্য defined

👉 কিন্তু এখন আমরা non-terminal state-এ আছি

✓ Solution:

👉 ব্যবহার করতে হবে:

Evaluation Function (Heuristic Function)

◆ Evaluation Function কী?

👉 এটি non-terminal state-এর value estimate করে

📌 Example (Chess):

- Pieces count

- King safety
- Board control

👉 এগুলো দিয়ে approximate score বের করা হয়



Final Summary (Exam-ready)

✓ Direct Minimax Steps:

1. DFS দিয়ে terminal state পর্যন্ত যাওয়া
2. Utility evaluate করা
3. Values propagate করা:
 - MIN $\rightarrow$ minimum
 - MAX $\rightarrow$ maximum
4. Best move select করা

✓ Complexity:

- Space $\rightarrow O(bd)$
- Time $\rightarrow O(b^d)$

! Problem:

- Time complexity খুব বেশি

✓ Solution:

- Depth-limited search

! New issue:

- Non-terminal states evaluate করতে হয়

✓ Solution:

- Evaluation (heuristic) function ব্যবহার করা
-



Static Board Evaluator (SBE)

◆ 1. Problem Recap (কেন SBE দরকার?)

👉 Minimax algorithm ideally পুরো game tree explore করে terminal state (win/lose/draw) পর্যন্ত যায়।

কিন্তু বাস্তবে (বিশেষ করে chess, go):

- Tree অনেক বড় 😬
- Time সীমিত 🕒

👉 তাই আমরা পুরো depth পর্যন্ত যেতে পারি না !

✓ Solution:

👉 আমরা একটা নির্দিষ্ট depth (m) পর্যন্ত search করে থেমে যাই

👉 তারপর ওই state-এর value estimate করি

📌 এই estimation করার জন্যই ব্যবহার হয়:

Static Board Evaluator (SBE)

♦ 2. SBE কী?

👉 **Static Board Evaluator (SBE)** হলো একটি function যা:

একটি game state (board position)-কে একটি numeric value দেয়

🎯 Goal:

👉 Board position কতটা ভালো বা খারাপ—তা measure করা

📊 Value Interpretation

Score

Meaning

Positive (+) AI-এর জন্য ভালো

Negative (-) Opponent-এর জন্য ভালো

0 Neutral

Static Board Evaluator (SBE)

* *A static board evaluation function estimates how good a board configuration is for the computer.*

- it reflects the computer's chances of winning from that state
- it must be easy to calculate from the board configuration

- For Example, Chess:

$$SBE = \alpha * materialBalance + \beta * centerControl + \gamma * \dots$$

material balance = Value of white pieces - Value of black pieces
(*pawn = 1, rook = 5, queen = 9, etc*).

◆ 3. SBE কী represent করে?

👉 SBE আসলে estimate করে:

“এই position থেকে AI জেতার chance কত?”

📌 অর্থাৎ:

- Higher score → বেশি chance to win
 - Lower score → কম chance
-

◆ 4. Important Properties of SBE

✓ Easy to compute

- খুব fast calculate করতে হবে

✓ Based on board configuration

- শুধুমাত্র current board দেখে decision নেয়

✓ Approximation

- exact না, but close estimation দেয়
-

◆ 5. Chess Example (সবচেয়ে গুরুত্বপূর্ণ অংশ)

👉 Chess-এ SBE সাধারণত multiple factors consider করে:

📌 Formula (General form):

$$\text{SBE} = \alpha \times \text{materialBalance} + \beta \times \text{centerControl} + \gamma \times \text{otherFactors}$$

◆ 6. Material Balance কী?

👉 এটি সবচেয়ে basic factor:

White pieces-এর value – Black pieces-এর value



Piece Values:

Piece	Value
-------	-------

Pawn 1

Knight 3

Bishop 3

Rook 5

Queen 9

Example:

ধরি:

- White: 1 Queen + 1 Pawn $\rightarrow 9 + 1 = 10$
- Black: 1 Rook $\rightarrow 5$

👉 Material Balance:

$$10 - 5 = +5$$

📌 Positive $\rightarrow$ AI (white) is winning

◆ 7. Other Factors

👉 শুধু pieces count না, আরও কিছু বিষয় important:

✓ Center Control

- Board-এর center control করা advantage দেয়

✓ King Safety

- King safe থাকলে ভালো

✓ Mobility

- কতগুলো legal move available

👉 এগুলো সব combine করে final score তৈরি হয়

◆ 8. Why “Static”?

👉 “Static” বলা হয় কারণ:

- এটি future moves consider করে না
- শুধু current board evaluate করে

🔥 Minimax tree-এ এটি leaf node (cut-off point)-এ ব্যবহার হয়

◆ 9. How SBE works with Minimax

👉 Process:

1. Minimax depth m পর্যন্ত যায়
 2. Terminal না হলে → SBE দিয়ে value calculate
 3. তারপর সেই value minimax rules অনুযায়ী propagate হয়
-



Final Summary (Exam-ready)

- ✓ SBE ব্যবহার হয় যখন full search possible না
- ✓ এটি board state-এর value estimate করে
- ✓ Positive → AI advantage
- ✓ Negative → opponent advantage
- ✓ Zero → neutral

✓ Properties:

- Fast
- Approximate
- Based on current board

✓ Chess example:

- Material balance
- Center control
- King safety

✓ Formula:

- Weighted combination of multiple factors
-

Designing a Good Static Board Evaluator (SBE)

👉 SBE এমনভাবে design করতে হবে যাতে AI সঠিক decision নিতে পারে—even যখন full game search করা সম্ভব না।

♦ 1. Order Terminal States Correctly (সবচেয়ে গুরুত্বপূর্ণ !)

মূল ধারণা:

Evaluation function যেন terminal states-গুলোকে ঠিকভাবে rank করে—ঠিক যেমন real utility function করে

✓ Rule:

- Win → Highest value
- Loss → Lowest value
- Draw → Neutral (0)

Example (Chess):

Win $\rightarrow +1000$

Loss $\rightarrow -1000$

Draw $\rightarrow 0$

কেন এটা জরুরি?

 যদি ranking ভুল হয়:

- AI ভুলভাবে বুঝবে কোন move ভালো
 - Winning move বাদ দিয়ে losing move choose করতে পারে
-

Bad SBE Example:

 যদি:

- Loss = +100
- Win = +50

 তাহলে AI loss-কে better ভাবে!

 Result:

 AI নিজেই হেরে যাবে

2. Must be Computed Quickly (Fast হওয়া লাগবে)

মূল ধারণা:

SBE function খুব দ্রুত calculate করা সম্ভব হতে হবে

! কেন?

👉 Game-৯:

- হাজার হাজার board state evaluate করতে হয়
- Time limit থাকে

👉 Slow function হলে:

- AI move নিতে দেরি করবে
- Real-time decision possible হবে না

✓ Solution:

👉 Use **lightweight heuristics** (simple calculation)

🧠 Example (Chess SBE):

$SBE = (\text{AI pieces value}) - (\text{Opponent pieces value})$

Piece values:

- Pawn = 1
- Knight = 3
- Bishop = 3
- Rook = 5
- Queen = 9

👉 এটা fast, কারণ:

- শুধু count করতে হয়
- future moves calculate করতে হয় না

◆ 3. Strong Correlation with Winning Chances

মূল ধারণা:

SBE যেন current position দেখে ভবিষ্যতে জেতার সম্ভাবনা ঠিকভাবে estimate করতে পারে

! কেন?

👉 যদি SBE inaccurate হয়:

- Weak position কে strong ভাবতে পারে
- Strong position কে ignore করতে পারে

👉 ফলে AI ভুল move নেবে

Example (Tic-Tac-Toe)

👉 SBE:

$SBE = (\text{AI-এর open winning lines}) - (\text{Opponent-এর open winning lines})$

✓ Good Situation:

- AI এর 2টা X একসাথে + 1 empty
→ Win করার chance বেশি → Positive score
-

! Dangerous Situation:

- Opponent এর 2টা O একসাথে + 1 empty
→ Opponent win করতে পারে → Negative score
-

⚠️ Bad SBE Example:

👉 যদি AI শুধু pieces count দেখে (threat ignore করে):

- Opponent winning move নিতে যাচ্ছে
- কিন্তু AI সেটা বুঝতে পারছে না

👉 Result:

➡️ AI block করবে না → Game lose করবে



Final Summary (Exam-ready)

✓ Good SBE-এর ৩টি মূল বৈশিষ্ট্য:

1. Correct Ordering

- Win > Draw > Loss
 - ভুল হলে AI ভুল decision নেবে
-

2. Fast Computation

- দ্রুত calculate করতে হবে
 - কারণ অনেক state evaluate করতে হয়
-

3. Strong Correlation

- Position দেখে winning chance accurately estimate করতে হবে
-

✓ Practical idea:

- Simple heuristics use করতে হবে
 - যেমন:
 - Material balance (chess)
 - Open lines (tic-tac-toe)
-

Evaluation Function & Minimax (Depth-limited + Heuristic)

◆ 1. Evaluation Function কী?

👉 Evaluation function (heuristic function) হলো এমন একটি function যা:

Non-terminal game state (intermediate position)-কে একটি numeric score দেয়, যাতে বোঝা যায় position কতটা ভালো।

📌 এটি ব্যবহার করা হয় যখন:

- পুরো game tree শেষ পর্যন্ত search করা সম্ভব না

👉 এই function-এ domain expert (game expert)-এর knowledge ব্যবহার করা হয়।

◆ 2. Evaluation Function as Heuristic

👉 Evaluation function মূলত একটি **heuristic**, কারণ:

- এটি exact solution না
- বরং approximate estimation দেয়

📌 অর্থাৎ:

“এই position কতটা ভালো হতে পারে”—তার একটি educated guess

◆ 3. Example: Tic-Tac-Toe Evaluation Function

$$f(n) = [\text{\# of 3-lengths open for me}] - [\text{\# of 3-lengths open for you}]$$

📌 এখানে:

- 3-length = complete row / column / diagonal
- Open 3-length = যেখানে এখনও win possible

Example:

ধরি:

- AI-এর জন্য 2টা winning line open
- Opponent-এর জন্য 1টা winning line open

👉 Then:

$$f(n) = 2 - 1 = +1$$

📌 Positive score → AI-এর জন্য ভালো position

◆ 4. Turing's Evaluation Function (Chess)

👉 Alan Turing chess-এর জন্য একটি simple evaluation function propose করেছিলেন:

$$f(n) = w(n) / b(n)$$

Where:

- $w(n)$ = white pieces-এর total value
- $b(n)$ = black pieces-এর total value

📌 Ratio যত বেশি → white advantage বেশি

◆ 5. Weighted Sum Evaluation Function

👉 Most modern evaluation functions use:

$$f(n) = w_1 * feat_1(n) + w_2 * feat_2(n) + \dots + w_k * feat_k(n)$$

 এখানে:

- $feat_1, feat_2, \dots, feat_k$ = different features
 - $w_1, w_2, \dots, w_k$ = weights (importance)
-

Chess Features Example:

- Piece count
- Piece placement
- Squares controlled
- King safety
- Mobility

 এগুলো combine করে final score তৈরি হয়

Deep Blue Example:

 Deep Blue-এর evaluation function-এ প্রায়:

~6000 features used হত 😬

 অর্থাৎ extremely complex heuristic model

◆ 6. Minimax with Evaluation Function (Depth-limited)

 Full minimax না করে আমরা use করি:

Depth-limited Minimax + Static Board Evaluator

Given Code Explanation (Conceptual)

Algorithm:

`minimax(node s, depth, limit)`

✓ Base Case:

- যদি terminal node হয় OR depth = limit
→ return staticEvaluation(s)
-

✓ Recursive Case:

1. Generate successors
 2. প্রতিটি successor-এর উপর minimax call
 3. Values collect করা
 4. তারপর:
 - MAX turn → maximum value return
 - MIN turn → minimum value return
-

🧠 Key Idea:

👉 Tree fully explore না করে:

- Limited depth পর্যন্ত explore করা হয়
- Leaf node-এ SBE ব্যবহার করা হয়

```
int minimax (Node s, int depth, int limit) {
    if (isTerminal(s) || depth == limit) //base case
        return(staticEvaluation(s));
    else {
        Vector v = new Vector();
        //do minimax on successors of s and save their values
        while (s.hasMoreSuccessors())
            v.addElement(minimax(s.getNextSuccessor(),
                                depth+1, limit));

        if (isComputersTurn(s))
            return maxOf(v); //computer's move returns max of
kids
        else
            return minOf(v); //opponent's move returns min of
kids
    }
}
```

◆ 7. How It Performs in Chess?

👉 Performance depends on depth of search:



Case 1: Lookahead ~4 pairs of moves (8 ply)

! Performance:

- Average players-এর কাছেও হেরে যাবে

👉 কারণ:

- Too shallow search
 - Future consequences miss করে
-



Case 2: Lookahead ~8 pairs of moves

✓ Performance:

- Human master-level performance

👉 Because:

- More accurate prediction
 - Better decision making
-

◆ 8. Important Limitations

! 1. Cannot search to the end

👉 Chess-এর search space এত বড় যে:

- Full search impossible
-

! 2. SBE is not perfect

👉 Evaluation function exact না:

- Approximation only
 - Some positions misjudge করতে পারে
-

! 3. If SBE were perfect...

👉 তাহলে:

No need for search !

➡ Just evaluate current position and pick best move



Final Summary (Exam-ready)

- ✓ Evaluation function = heuristic function
- ✓ It estimates the value of non-terminal states
- ✓ Uses domain expert knowledge

✓ Examples:

- Tic-Tac-Toe:
 - Open winning lines difference
- Chess:
 - Material ratio (Turing)
 - Weighted sum of features

✓ Modern evaluation:

- $f(n)$ = weighted sum of multiple features
- Deep Blue used ~6000 features

✓ Minimax with evaluation:

- Depth-limited search
- Terminal or cutoff → SBE used

✓ Performance:

- Shallow search → weak performance
- Deeper search → stronger AI

✓ Limitation:

- Cannot search full tree
- Evaluation function not perfect

1. Alpha-Beta Pruning Idea

Minimax এ সব branch explore করা লাগে, কিন্তু—

- 👉 অনেক branches আছে যেগুলো **optimal opponent** খেললে কখনোই **choose** হবে না
- তাই সেগুলো **prune (cut off)** করা যায়

💡 Goal:

Search tree কমানো, কিন্তু final answer unchanged রাখা

2. DFS Traversal + Two Values

Alpha-Beta Pruning Idea



- **Beta cutoff** pruning occurs when **maximizing** if *child's alpha* $\geq$ *parent's beta*
Why stop expanding children?
opponent won't allow computer to take this move
- **Alpha cutoff** pruning occurs when **minimizing** if *parent's alpha* $\geq$ *child's beta*
Why stop expanding children?
computer has a better move than this

Alpha-beta pruning DFS (Depth First Search) দিয়ে করা হয়।

♦ Alpha (α)

- Maximizing player (computer) এর best value so far
- Initially: $-\infty$
- Represents: **lower bound**

👉 Meaning: “আমি অন্তত এতটা ভালো outcome পেতে পারি”

♦ Beta (β)

- Minimizing player (opponent) এর best value so far
- Initially: $+\infty$
- Represents: **upper bound**

👉 Meaning: “Opponent আমাকে এর চেয়ে বেশি যেতে দিবে না”

Alpha-Beta Pruning Idea



- **Beta cutoff** pruning occurs when **maximizing** if *child's alpha* $\geq$ *parent's beta*
Why stop expanding children?
opponent won't allow computer to take this move
- **Alpha cutoff** pruning occurs when **minimizing** if *parent's alpha* $\geq$ *child's beta*
Why stop expanding children?
computer has a better move than this

♦ 3. Pruning Conditions

✓ **Beta Cutoff (Max node)**

Condition:

$$\alpha \geq \beta$$

👉 তখন আর explore করার দরকার নেই

কেন?

- Opponent (min player) এই path এ কখনো allow করবে না
- কারণ opponent already better option আছে

➡ তাই remaining children prune করা হয়

✅ Alpha Cutoff (Min node)

Condition:

$$\alpha \geq \beta$$

👉 একই condition, কিন্তু context min node

কেন?

- Computer already better move has
- So opponent path is useless

➡ pruning করা যায়

৩. শর্তগুলো মনে রাখার সহজ উপায়

Cutoff Type	Condition	Who prunes?	Why?
Beta Cutoff	$Child's \alpha \geq Parent's \beta$	MAX Player	Opponent will never reach this high score.
Alpha Cutoff	$Parent's \alpha \geq Child's \beta$	MIN Player	You already have a better move elsewhere

◆ 5. Effectiveness Depends on Ordering

Alpha-beta pruning এর performance খুব depend করে:

👉 **successors** কোন **order** এ **explore** করা হচ্ছে

◆ Best Ordering:

- Best moves আগে explore করা হয়

👉 Result:

- Maximum pruning
 - Minimum computation
-

◆ Worst Ordering:

- Worst moves আগে explore হয়

👉 Result:

- No pruning
 - Same as minimax (brute force)
-

◆ 6. Complexity Improvement

Without pruning:

$$O(b^d)$$

With alpha-beta (best case):

$$O(b^{(d/2)})$$

👉 Equivalent to:

- branching factor becomes $\sqrt{b}$

Mathematical intuition:

$$(\sqrt{b})^d = b^{(d/2)}$$

7. Practical Behavior

👉 In real applications:

- Performance usually close to best case
 - Because good move ordering techniques are used
-

8. Example: Chess

- Normal branching factor: ~35
- With alpha-beta pruning effectively: ~6

👉 Benefits:

- Much deeper search possible
 - Makes computer chess strong
-

9. Summary (Exam Ready)

- Alpha-beta pruning is an optimization of minimax
 - It eliminates branches that cannot affect final decision
 - Uses two parameters:
 - Alpha → best value for maximizer (lower bound)
 - Beta → best value for minimizer (upper bound)
 - Pruning occurs when:
 - $\alpha \geq \beta$
 - Depends heavily on move ordering
 - Reduces complexity from $O(b^d)$ to $O(b^{(d/2)})$ in best case
 - Widely used in games like chess
-

1. Horizon Effect কী?

👉 Horizon Effect হলো এমন একটি সমস্যা যেখানে:

Computer limited depth পর্যন্ত search করে, কিন্তু সেই সীমার ঠিক পরেই কোনো বড় disaster লুকিয়ে থাকে।

One-Line Intuition Summary

- **Horizon effect:**
“I can’t see far enough, so I might miss future danger.”
 - **Quiescence search:**
“Don’t evaluate in a chaotic situation—wait until things settle.”
 - **Secondary search:**
“Don’t trust the first best move blindly—verify it deeper.”
-

Example:

- Computer দেখে:
 - “আমি queen capture করতে পারছি” ✓
- কিন্তু কয়েক move পর:
 - opponent checkmate করে দেয় ✗

👉 সমস্যা:

Computer ওই future disaster দেখতে পায় না, কারণ সেটা তার search depth এর বাইরে।

Intuition:

👉 Computer এর একটা “horizon” (দূর দেখার সীমা) আছে

👉 Horizon এর বাইরে কিছু important ঘটনা ঘটলে সে miss করে যায়

2. কেন এটা ঘটে?

কারণ:

- Minimax/Alpha-beta search depth limited

- Evaluation function exact না (heuristic)
 - Important tactical sequences depth এর বাইরে থাকতে পারে
-

3. Problem:

- 👉 Short-sighted decisions নেয়
 - 👉 Local best move globally worst হতে পারে
-

4. Solutions

এই problem avoid করতে দুইটা technique use করা হয়:

(1) Quiescence Search

Idea:

Search depth শেষ হলেও, যদি position “unstable” বা “noisy” হয়, তাহলে আরও deep search করা হয়।

When to apply?

- Frequent changes in evaluation
 - Capture / check / tactical moves present
 - Position not “quiet”
-

Goal:

- 👉 Reach a **quiescent (stable)** position
 - 👉 যেখানে evaluation আর suddenly change হবে না
-

Example:

- Capture sequences

- Checks
- Forced moves

👉 এগুলো চলতে থাকে যতক্ষণ position stable না হয়

✅ (2) Secondary Search

◆ Idea:

Two-stage verification:

Step 1:

- Depth d পর্যন্ত best move খুঁজে বের করা

Step 2:

- সেই move কে verify করতে extra k steps lookahead করা
-

◆ If move is not good after verification:

👉 Next best move try করা হয়

👉 আবার verify করা হয়

◆ Process Flow:

1. Best move (depth d)
 2. Look ahead k steps beyond d
 3. If still good → accept
 4. If not → next candidate move evaluate
-

◆ 5. Comparison

Feature

Quiescence Search

Secondary Search

Approach	Extend search in unstable positions	Verify best moves further
Focus	Stability of position	Move correctness
Usage	Tactical noise handling	Move validation
Trigger	Position volatility	After best move selection

6. Why These Matter?

Without these:

- Minimax + alpha-beta may choose dangerous moves
- Horizon effect causes blunders

With these:

- More realistic evaluation
 - Avoid catastrophic mistakes
 - Better tactical awareness
-

7. Exam Summary (Short Answer)

- Horizon effect occurs when a game tree search cannot see beyond a limited depth and misses important future consequences.
- It leads to short-sighted decisions such as capturing a piece that later results in loss.
- To overcome this:
 - **Quiescence search** extends search in unstable positions until a quiet (stable) state is reached.
 - **Secondary search** verifies the best move by looking further ahead and re-evaluating if necessary.
- These techniques reduce tactical errors in game-playing AI.
